WEEK 3 ASSIGNMENT

Comparing Supervised Learning Models

Fayz Liaqat | 23 October 2026

GitHub: github.com/fayzliaqat/week3-supervised-models

01 Dataset and preparation

I continued with **Loan Predication** from Kaggle (amit parjapat): **614 rows and 13 original columns**. The target is historical loan approval: **1 = approved, 0 = not approved**. There are 422 approved applications (68.73%) and 192 not approved (31.27%), so this is imbalanced binary classification.

I found 149 missing cells across seven input columns, no missing targets, and no exact duplicate rows. I dropped the application ID and kept the two Week 2 features, **totalIncome** and **hasCoapplicantIncome**.

The outer split was **491 training / 123 test rows**, stratified with random_state=42. Inside training, I used **392 tuning / 99 validation rows**. Numeric inputs used median filling and StandardScaler; text used mode filling and OneHotEncoder. Seven numeric and six categorical features produced **22 processed inputs**.

For tuning, preprocessing was fitted only on the 392 tuning rows. After selection, a fresh preprocessor was fitted on all 491 training rows. Validation and test rows were only transformed, so their statistics did not guide fitting.

02 Tuning and choosing a model

KNN: I tried odd K values from 1 to 15. **K = 5** had the highest validation accuracy, **81.82%**. **Decision Tree:** I tried depths 2, 3, 5, 10, and None. Depth **2** scored **78.79%**; tied settings kept the shallower tree.

The unrestricted tree reached depth 14: training accuracy was 100.00%, but validation accuracy was only 75.76%. This gap suggests overfitting. A deeper tree can memorize small details instead of learning patterns that carry over.

For this exercise, I would use **KNN**. It was **selected before test evaluation** using the highest validation macro-F1, **0.7451**. Macro-F1 gives both classes equal weight. Ties favor approval precision, then accuracy. I kept that decision after seeing test results.

03 Final test comparison

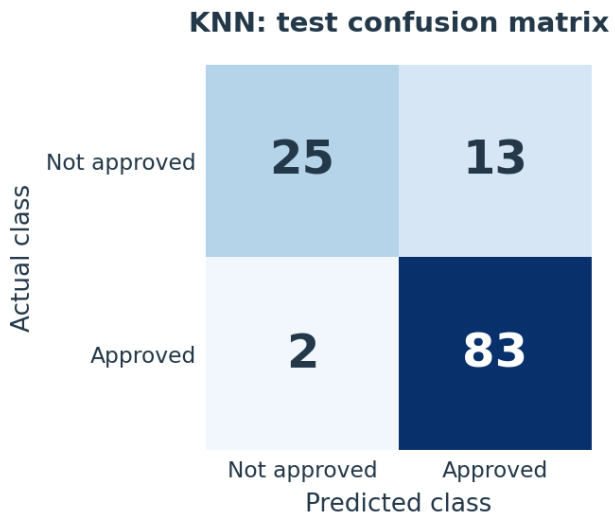
Model	Accuracy	Precision	Recall	F1	Macro-F1
KNN	0.8780	0.8646	0.9765	0.9171	0.8432
Decision Tree	0.8455	0.8367	0.9647	0.8962	0.7973
Logistic Regression	0.8618	0.8400	0.9882	0.9081	0.8147

All three classifiers used the same processed training/test rows. Precision, recall, and F1 refer to approved (1). The highlighted model was selected on validation data, not on test scores.

The majority-class baseline scored **69.11%** accuracy, but rejection F1 was **0.0000** and macro-F1 was **0.4087**. Accuracy alone hides this weakness.

MODEL MISTAKES AND WHAT I LEARNED

04 Understanding the selected model's errors



TN = 25: correctly predicted not approved.

FP = 13: predicted approved, but the recorded class was not approved.

FN = 2: predicted not approved, but the recorded class was approved.

TP = 83: correctly predicted approved.

The model made 15 mistakes among 123 test applications. It identified 25 of 38 not-approved cases (65.79% recall for class 0).

Rows are actual classes and columns are predicted classes. A false approval here is disagreement with a historical decision; it does not prove the applicant would default.

05 Which mistakes matter more?

For a hypothetical approval-screening exercise, I would assume a false approval is more costly and pay close attention to **approval precision**. For KNN, it is **0.8646**. This is an assumed priority, not a measured financial cost. Rejection errors and both class F1-scores still matter. These labels describe approval decisions, not repayment or creditworthiness.

06 Extra comparison: Linear Regression

The opening goal also mentions Linear Regression. I fitted it separately on the same training set and used a **fixed 0.5 threshold** to turn continuous scores into class predictions. It was excluded from the main three-classifier selection.

Accuracy	Precision	Recall	F1	Macro-F1
0.8618	0.8400	0.9882	0.9081	0.8147

Raw-score metrics: **MAE 0.2575**, **RMSE 0.3413**, and **R-squared 0.4543**. Scores ranged from -0.0616 to 1.0180. They are continuous scores, not probabilities, and can fall outside 0 to 1.

07 Problems faced and what I learned

The main challenges were keeping tuning separate from final testing and interpreting Linear Regression in a classification task. I learned that choosing K or depth from test scores also leaks information, even if the scaler itself is fitted correctly.

I also learned to check minority-class results and a simple baseline instead of trusting accuracy alone. The notebook ran top to bottom, and the final inputs had no missing or infinite values. Logistic Regression probabilities for five test rows are included in the notebook.

One small validation split can give noisy results. Cross-validation is a useful next step. This exercise has no external validation or fairness assessment and does not establish suitability for real lending decisions.